

VCU College of Engineering

Project #44

Game Engine PAL: Embedded-Friendly Game Engine

Project Proposal

Prepared for

Olivier Henley
AdaCore

By

Brenna Wheaton, Brandon Hening, and Benjamin Karp

Under the Supervision of

Dr. Ahmet Sonmez

October 9, 2026

Executive Summary

In collaboration with AdaCore, our Game Engine PAL project is an integration project that uses Ada PAL, a rendering and input layer, and ECS PAL, a scene management layer, and turns them both into a functional game engine that shows that the two components can work together as a cohesive, portable system. This is done by delivering both a reusable engine architecture and a playable 2D game that is able to run on both desktop and embedded platforms. This will not be done from scratch, but by using this project to integrate, extend, and validate the capabilities of these two existing PAL components.

The initial challenge of this project is to create the game engine and ensure that it is both lightweight and able to run a 2D game. To do this, this means integrating the rendering, input-handling, and the ECS scene management while providing the core infrastructure required to have it run a complete game. A key requirement for this task includes building the gameplay loop, consisting of a fixed-timestep scheduler with interpolated rendering, frame timing via Ada.Real_Time with no floating point time calculations, and protections against spiral-of-death on the accumulator. The integration will also include using the QOI asset pipeline along with the QOI Alire crate, which will be loaded on both desktop and targets that are embedded.

The 2D game to be delivered will feature multiple different assets, such as multiple entity types with distinct component sets, an input-driven player system, at least one autonomous system such as enemies or physics, and win and lose conditions that are managed through in-game state transitions. This particular project will not feature a custom ECS or changes to Ada PAL's rendering in any way, as the main focus of this project will be to focus on successful integration between these dependencies and guaranteeing successful composition.

Our team for this project will mainly focus on establishing the integration architecture and implementing the dependency integration. This will then evolve into working on rendering and scene management, and eventually focus on developing the game itself. Future objectives will include validating the state transitions, developing and testing identical source code, a performance profile, ensuring SPARK proof of Game.Loop, and publishing the Alire crate. This will also include a final report defending the architecture and dependency boundaries.

The completion of the Game Engine PAL project will show that Ada PAL and ECS PAL can be integrated together into a working, stable game engine that can be built off of the capabilities of both. A game that is created on this engine, as well as its development cycle, will be able to run effectively and efficiently. With the documentation, verification, and a playable game in hand, this project will be a reference for developers going into portable game development and show that software components in Ada are able to be used to build these games efficiently.

Table of Contents

Section	Page
Executive Summary	2
Section A. Problem Statement	4
Section B. Engineering Design Requirements	4
B.1 Project Goals (Client Needs)	4
B.2 Design Objectives	5
B.3 Design Specifications and Constraints	6
Section C. Scope of Work	7
C.1 Deliverables	8
C.2 Milestones	9
C.3 Resources	10
Appendix 1: Project Timeline	12
Appendix 2: Team Contract	13
References	16

Section A. Problem Statement

Software is used across a wide range of industries, from everyday applications to complex fields such as healthcare or the military, and there are many different ways to use it and problems it solves. AdaCore is a company that engineers software, specifically for development services for Ada and SPARK along with C/C++ and others. They make software tools for a wide variety of industries such as aerospace, transportation, defense, and medicine, and aim to provide more reliable, safe and secure software.

Our present project is sponsored by AdaCore and centers around building a working and playable game engine by combining and integrating Ada PAL, which handles the rendering and input events routed to game systems, with ECS (Entity-Component-System) PAL, which handles the scene management and entity lifecycle tied to game state transitions. Combining Ada PAL and ECS PAL is necessary because they do not natively work with each other, meaning one library's code could leak into the other's internals or break boundaries, along with having potential missing dependencies or dependencies behaving in unintended ways. Along with the game engine, the 2D game itself must be playable on both desktop and bare-metal embedded systems, meaning that dynamic memory allocation cannot be used in the update loop to have both systems able to compile. Along with the working game engine and playable 2D game, this project sets out to prove that Ada based game engines do not suffer from the same frame timing bugs which are common in C++ engines, due to its floating-point time accumulating drift and overflowing. These issues can lead to physics running faster at high frame rates, leading to bugs, or the game death spiraling when a frame takes too long to load. These bugs would be structurally impossible due to Ada's fixed-point time types, which help to stabilize game physics and logic. This project was designed in mind to benefit potential future Ada developers by allowing them to use it as a template for building games with the existing libraries, and had been worked on previously by the company as we have received some already working source code.

Section B. Engineering Design Requirements

The engineering design requirements for Game Engine PAL are based on AdaCore's project specifications and the limitations of real-time embedded systems.

The project focuses on reliable performance, organized software design, compatibility across different platforms, and formal verification. The following goals, objectives, and constraints will be used to determine whether the final design meets the project requirements.

B.1 Project Goals (i.e. Client Needs)

The primary goal of Game Engine PAL is to demonstrate how existing Ada software components can work together to create a functional real-time game engine.

Below are the project goals identified by AdaCore:

1. Develop an embedded-friendly game engine integration layer that combines Ada PAL and ECS PAL.
2. Establish clearly defined software architecture boundaries between the game engine, rendering and input services, and entity-component scene management.
3. Demonstrate reliable real-time simulation through a fixed-timestep game loop.
4. Support execution on both desktop and bare-metal embedded platforms without requiring changes to game source code.
5. Demonstrate the architecture through a complete playable 2D game.
6. Apply formal verification techniques to critical game-loop functionality.
7. Evaluate execution stability and timing performance on embedded hardware.
8. Publish the completed software as an Alire crate for the Ada development community.

The main focus of this project is to combine existing systems into a working game engine rather than build a completely new one from scratch.

B.2 Design Objectives

The design objectives mainly ensure that both the engine and the game are fully functional. Each objective will describe what the completed system must accomplish and provide a criterion for evaluating project success. These objectives are met through implementation, testing, performance measurement, and formal analysis:

1. The design will provide a functional game loop that executes simulation updates at a fixed timestep while supporting interpolated rendering.
2. The design provides reliable frame timing using Ada.Real_Time without relying on floating-point time calculations, and will enforce an accumulator preventing spiral-of-death behavior.
3. The design will integrate Ada PAL and ECS PAL so that the input events reach the appropriate systems and so that they remain consistent with game state transitions.
4. The design will provide a functional QOI asset that loads images using the QOI Alire crate on both desktop and embedded targets.
5. The design will deliver a 2D game with multiple entity types, an input-driven player system, at least one autonomous system, and functional win-lose conditions.
6. The design will provide both engine architecture and engine implementation with Ada PAL and ECS PAL, and will provide an embedded performance profile measuring the loop, the asset pipeline, and the subsystem boundaries.
7. The design will provide distributed deliverables including defending the architecture and dependency boundaries, as well as the final Alire crate published to the community index.
8. Proof of the first playable game slice will be delivered during the Fall 2026 semester, and the complete implementation and final delivery of the project will be delivered during the Spring 2027 semester.

B.3 Design Specifications and Constraints

The design must meet AdaCore's requirements for functionality, performance, portability, and software architecture.

Design Requirement	Constraint Type	Target/Specification
Fixed-timestep game loop	Functional	Ada.Real_Time and fixed-point timing
Simulation update rate	Performance	Stable 60 Hz target
Timing accumulation	Reliability	Bounded accumulator
Update execution	Reliability	Finite updates per tick
Interpolation	Functional	Factor between 0.0 and 1.0
Rendering and input	Architecture	Ada PAL
Entity management	Architecture	ECS PAL
Asset loading	Functional	QOI asset pipeline
Memory allocation	Embedded	No dynamic allocation during update loop

Platform compatibility	Portability	Same source on desktop and embedded
Formal verification	Verification	GNATprove Silver target
Stability	Performance	Five minutes without crashes or timing failures
Package distribution	Deliverable	adacore_gameengine Alire crate

The fixed-timestep scheduler will limit the number of simulation updates that can build up when frames are delayed. Instead of using floating-point values to track time, the scheduler will use fixed-point values with a defined level of precision.

The engine must also keep the game application separate from the libraries it depends on. Rendering and input handling will go through Ada PAL, while ECS PAL will handle entities and components.

To make sure the engine can run on embedded hardware, the update loop will avoid allocating memory while the game is running. Entity and component storage will be set up during initialization using ECS PAL.

Other constraints include the two-semester project timeline, the availability of required libraries, our development resources, and the limitations of the embedded hardware we choose.

The final design will be evaluated through unit testing, integration testing, performance testing on embedded hardware, and formal verification.

Section C. Scope of Work

The scope of Game Engine PAL includes designing, developing, testing, verifying, and documenting a game engine that can run on embedded systems.

The project will focus on combining existing Ada libraries rather than building an entirely new game engine from scratch. Our team will develop a fixed-timestep game loop, integrate rendering and input handling, connect the entity-component systems, create an asset-loading pipeline, and build a playable 2D game to demonstrate how everything works together.

Development will be divided into several milestones starting with the engine's architecture and desktop integration, followed by game development, embedded testing, formal verification, and final delivery.

Throughout the project, our team will meet regularly with our AdaCore technical mentor, Olivier Henley, and faculty advisor, Dr. Ahmet Sonmez, to discuss progress, review our work, and address any issues.

The following features are outside the scope of this project:

- Developing a general-purpose game engine
- Making direct graphics calls outside of Ada PAL
- Implementing an audio system
- Creating a level editor
- Developing a scripting system

By excluding these features, our team can focus on meeting AdaCore's main project requirements and completing the engine within the two-semester timeline.

C.1 Deliverables

The project deliverables will include the completed software, technical documentation, testing and verification results, and demonstrations showing how the system works.

Technical Deliverables

1. Game engine architecture documentation describing subsystem boundaries, the fixed-timestep loop, and asset management.
2. A functioning game engine integration layer combining Ada PAL and ECS PAL.
3. A fixed-timestep scheduler using Ada.Real_Time and fixed-point timing.
4. A QOI-based asset-loading pipeline.
5. A complete playable 2D game containing player-controlled and autonomous game systems.
6. Desktop and embedded builds using identical game source code.
7. Embedded hardware performance measurements, including frame timing and memory usage.
8. SPARK and GNATprove verification results for the Game.Loop package.
9. Unit testing, integration testing, and stability testing results.
10. The adacore_gameengine Alire crate submitted to the community index.
11. Final technical documentation describing the architecture, implementation, dependency boundaries, performance, and verification results.

Academic Deliverables

- Team Contract (September 2026)
- Project Proposal (October 2026)
- Fall Design Poster (November 2026)
- Preliminary Report (December 2026)

- Expo Abstract (March 2027)
- Expo Design Poster (March 2027)
- Final Report (April 2027)

Potential Risks and Mitigation

- Dependency Availability: The project relies on Ada PAL and ECS PAL, which may need updates throughout development. If either library is unavailable, temporary stub interfaces can be used until the necessary components are ready.
- Embedded Compatibility: Desktop and embedded systems have different requirements, which could cause issues when compiling or running the game. The team will identify these differences early and test the game on embedded hardware throughout development.
- Real-time Performance: Inconsistent frame timing could affect how smoothly the game runs. To prevent this, the game loop will limit accumulated time, and the team will use testing and formal verification to check its performance.
- Integration Complexity: Combining different software components can be challenging, especially if their responsibilities are not clearly defined. The team will review architecture and document how the components interact to keep the engine, platform, and game logic separate.
- Time Constraints: The project must be completed within two semesters. To stay on schedule, the team will divide the work into smaller milestones and focus on completing the main engine, playable game, embedded compatibility, and verification.

C.2 Milestones

The project will follow AdaCore's proposed two-semester schedule. The work will be divided into smaller milestones to help our team stay organized, track progress, and make sure we are meeting project requirements.

Milestone	Description	Target Completion
Architecture and Dependency Integration	Define subsystem boundaries and implement initial game loop	Fall 2026, Weeks 1–4
Desktop Engine Integration	Integrate Ada PAL, ECS PAL, and QOI asset loading	Fall 2026, Weeks 5–8

First Playable Game Slice	Implement player controls, autonomous behavior, and rendering	Fall 2026, Weeks 9–12
Embedded Port	Execute game on embedded hardware and begin profiling	Fall 2026, Weeks 13–16
Complete Game Implementation	Implement game states and win/lose conditions	Spring 2027, Weeks 17–22
Formal Verification and Stability Testing	GNATprove verification and embedded testing	Spring 2027, Weeks 23–26
Documentation and Packaging	Complete documentation and prepare Alire crate	Spring 2027, Weeks 27–29
Final Delivery	Final demonstration, report, and crate submission	Spring 2027, Weeks 30–32

The schedule may change depending on the availability of required libraries, feedback from our mentor, and any issues that come up during development and testing.

C.3 Resources

Game Engine PAL will mainly require software development tools, existing Ada libraries, and access to computers and embedded hardware for development and testing.

Software and Development Resources

- Ada programming language and GNAT compiler toolchain.
- Alire package manager.
- Ada PAL rendering and input library.
- ECS PAL entity-component management library.
- QOI Alire crate for image asset loading.

- SPARK and GNATprove for formal verification.
- Visual Studio Code or an equivalent Ada-compatible development environment.
- Git and a shared source-code repository for version control.
- Build and debugging tools for desktop and embedded targets.

Hardware Resources

- Development Computers: Personal laptops or desktop computers used to write, compile, and test the Ada software.
- Embedded Hardware: An Ada PAL-compatible embedded device to test the game outside of a desktop environment. The specific hardware will be determined with the project mentor.
- Programming and Debugging Equipment: Any cables, interfaces, or debugging tools required by the selected embedded device.

Human Resources

The project team consists of three VCU Computer Science students:

- Brenna Wheaton: Systems and Manufacturing Engineer
    - Responsible for backend software development and frontend design, as defined in the team contract.
- Brandon Hening: Testing Engineer and Financial Manager
    - Responsible for testing-related data collection, statistical analysis, budget oversight, and purchasing coordination.
- Benjamin Karp: Project Manager and Logistics
    - Responsible for project schedules, meeting agendas, planning, and coordination with the faculty advisor.

Project Mentorship

- Olivier Henley of AdaCore will serve as the technical mentor and primary sponsor contact.
- Dr. Ahmet Sonmez will serve as the VCU faculty advisor.

Budget Considerations

Our team has a budget of $1500 for this project. Since the project is primarily software-based, we expect minimal expenses. We may need to purchase embedded hardware or debugging equipment, depending on what resources are available. Any necessary purchase will be discussed with our project mentor and faculty advisor before using the budget.

Appendix 1: Project Timeline

The following timeline summarizes the proposed development activities for the 2026–2027 academic year.

Academic Period	Planned Activities
September–October 2026	Architecture design and dependency integration
October–November 2026	Desktop engine, asset pipeline, and input integration
November–December 2026	Playable prototype and embedded integration
January–February 2027	Complete gameplay and state management
February–March 2027	Formal verification and stability testing
March–April 2027	Documentation, testing, and Alire packaging
April–May 2027	Final presentation and delivery

This timeline is based on AdaCore's proposed implementation plan and will be refined as development progresses.

Appendix 2: Team Contract

Step 1: Get to Know One Another. Gather Basic Information.

Team Member Name	***Strengths each member bring to the group***	***Other Info***	***Contact Info***
Brenna Wheaton	- Organization - Front End programming		757-300-7278 wheatonbl@vcu.edu
Brandon Hening	- Statistical background - Effective constructive criticism - Team communication		804-539-4832 heningbm@vcu.edu
Benjamin Karp	- Communicating - Back end / ERD design		804-213-6367 karpbf@vcu.edu

Other Stakeholders	***Notes***	***Contact Info***
Olivier Henley	*Adacore, mentor/client*	henley@adacore.com
Ahmet Sonmez	Faculty Advisor	sonmeza@vcu.edu

Step 2: Team Culture. Clarify the Group's Purpose and Culture Goals.

Culture Goals	***Actions***	***Warning Signs***
Commitment	- Show up time for meetings - Be prepared - Complete your tasks - Respond in a reasonable amount of time	-Not showing up to classes/meetings without communicating -Not responding to messages in discord in a reasonable amount of time -Other team members can reach out if there are any miscommunications/issues
Trust	- Rely on each other - Be accountable	–Not doing their part of the work in a reasonable amount of time -Other team members can ask them if they need help/reach out to advisor if they still don't do any work
Constructive Communication	- Be direct, but open - Be respectful and Positive	-Being rude or demeaning if there is an issue/mistake -Be polite and say things in a kinder way
Collaboration	- Work together - Ask for help - Decide as a group	-One person making all of the decisions/not listening to other team member's input -Discuss together and come to a solution that everyone can work with

Step 3: Time Commitments, Meeting Structure, and Communication

Meeting Participants	***Frequency*** ***Dates and Times / Locations***	***Meeting Goals*** ***Responsible Party***
Students	Mondays at 3:30pm	Go over work needed to be completed for the week Arranged by project manager

Students, Client, Faculty Advisor	Wednesdays at 3:30pm / Google Meet	Check in with client. Arranged by client
Students, Faculty Advisor	Fridays / Time varies weekly	Review work completed Arranged by project manager

Step 4: Determine Individual Roles and Responsibilities

Team Member	***Role(s)***	***Responsibilities***
Brenna Wheaton	Systems and Manufacturing Engineer	- Backend Software Development - Frontend Design
Brandon Hening	Testing Engineer, Financial Manager	- Overseeing project budget - Managing purchases - Acquiring data from project testing - Presenting statistical findings to other team members
Benjamin Karp	Project Manager, Logistics	- Develop schedules and agendas - Planning - Coordinate with advisor

Step 5: Agree to the above team contract

Team Member: Brenna Wheaton *Signature: Brenna Wheaton*

Team Member: Benjamin Karp *Signature: Benjamin Karp*

Team Member: Brandon Hening *Signature: Brandon Hening*

References

AdaCore. (2025). *Formal verification with GNATprove*. In *SPARK user's guide* (Version 25.1). https://docs.adacore.com/R/docs/gnat-25.1/spark2014/html/spark2014_ug/en/gnatprove.html

AdaCore. (2026, July 29). *Game Engine PAL: Embedded-friendly game engine* [Senior design capstone project specification].

AdaCore. (n.d.). *Introduction*. In *Introduction to Ada*. https://learn.adacore.com/courses/intro-to-ada/chapters/introduction.html

Godot Engine. (n.d.). *Introduction to physics interpolation*. https://docs.godotengine.org/en/stable/tutorials/physics/interpolation/physics_interpolation_introduction.html

Henzinger, T. A., & Kirsch, C. M. (2002). *The embedded machine: Predictable, portable real-time programming*. https://web.cs.ucla.edu/~palsberg/course/cs239/S04/papers/HenzingerKirsch02.pdf

IEEE. (2024). *Untrusted code compartmentalization for bare metal embedded systems*. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*. https://ieeexplore.ieee.org/document/10745829/

Koopman, P. (1995). *Challenges in embedded systems research & education*. Carnegie Mellon University. https://users.ece.cmu.edu/~koopman/embedded/challenges.pdf

Thapa, B., et al. (2026). *Embedded Rust or C firmware? Lessons from an empirical study*. arXiv. https://arxiv.org/html/2604.25679v2

Wang, Y., et al. (2023). A survey of the security analysis of embedded devices. *Sensors, 23*(22), Article 9180. https://pmc.ncbi.nlm.nih.gov/articles/PMC10674897/